IT4010 Research Project

Proposal Report

Component 2 . Dynamic Model Updating with Concept Drift Detection

Student Name	Shalon Anton Fernando
Student ID	IT22362544
Project ID	J26-SE-335
Supervisor	Ms. Hansi De Silva
Co-Supervisor	Mr. Eishan Weerasinghe
Specialization	Software Engineering (SE)
Focus Area	Online ML and MLOps Lifecycle Management

1. Knowledge Gap and Novel Solution

Most production AI systems work like a photograph. You take the picture once, frame it nicely, and hang it on the wall. Six months later the world has changed but the photograph has not. The anomaly detection model you trained in January still thinks January normal traffic patterns are normal. It has no idea that your engineering team deployed a major service in February that doubled baseline CPU usage. Now it flags legitimate behaviour as anomalies forty times a day and your operations team has learned to ignore every alert. That is not a self healing system. That is a very expensive smoke alarm that cried wolf until everyone stopped listening.

This problem has a name in the machine learning literature. It is called concept drift. Concept drift occurs when the statistical relationship between input features and the target distribution shifts over time. For a microservices anomaly detection system this happens constantly. New deployments change baseline resource usage. Seasonal traffic spikes reshape normal network patterns. Business logic changes alter request rate distributions. A model trained before these changes progressively degrades, producing more false positives and missing genuine failures.

The knowledge gap is this. Most organisations know about concept drift. They solve it by retraining models on a fixed schedule by hand. Someone writes a cron job, someone else approves the new model, and the organisation hopes the drift was not too bad in the weeks between retrains. This is reactive, manual, and completely unsuitable for a system that is supposed to be self healing.

No existing open framework combines automated drift detection with immediate model rollback and a full versioned audit trail in a production microservices context. That is the gap. This component fills it by building the automated continuous lifecycle management layer that keeps the anomaly detection model accurate indefinitely without any human intervention.

The novel contribution is threefold. First, the system detects drift in the anomaly score stream itself using ADWIN (Adaptive Windowing), catching distribution shifts before they manifest as missed detections at the service level. Second, the system does not just detect drift and report it. It automatically rolls back to the last verified stable model version, restoring detection quality immediately. Third, every model state transition is versioned and audited in MLflow with full context, giving the operations team a complete forensic record of every adaptation the system made.

2. Existing Systems and Related Work

The dominant tools in the microservices observability space, Prometheus and Grafana, excel at collecting and visualising metrics but have no predictive intelligence whatsoever. They report that something went wrong after it went wrong. Grafana cannot retrain itself. Prometheus has no concept of a model version. They are beautifully engineered reactive tools that solve a different

problem than the one addressed here (Soldani et al., 2018).

Commercial AIOps platforms like Dynatrace and Datadog have begun incorporating ML based anomaly detection. However these platforms are proprietary black boxes. You cannot inspect their model architecture, trigger a rollback to a previous model version, or plug in your own drift detector. Their adaptation mechanisms, if they exist at all, are not documented in any way that allows scientific evaluation or comparison. For a research component that must demonstrate measurable improvement over a baseline, a black box is not a viable foundation.

The river machine learning library (Montiel et al., 2021) provides excellent streaming ML primitives including Half Space Trees and the ADWIN drift detector. River solves the online learning problem elegantly. What river does not provide is any model lifecycle management. It has no concept of versioning, no rollback mechanism, and no audit trail. If you detect drift in river, you know about it. What you do next is entirely your problem.

MLflow (Zaharia et al., 2018) provides model versioning, experiment tracking, and a model registry with stage management. MLflow is an excellent tool for the lifecycle management side of the problem. What MLflow does not provide is any automated trigger mechanism. It does not watch your model's production behaviour and decide when to initiate a rollback. That decision must come from somewhere else.

The gap between these two ecosystems, river's online learning and drift detection on one side and MLflow's lifecycle management on the other, is precisely where this component lives. The contribution is the integration layer that uses drift detection as a signal to drive automated lifecycle transitions in a versioned and auditable way.

The foundational work on ADWIN by Bifet and Gavalda (2007) established the statistical basis for adaptive windowing as a drift detector. Their work proved that ADWIN provides rigorous false positive rate guarantees through the Hoeffding bound. This research is seventeen years old and still has no widely adopted production implementation in a model lifecycle context. That tells you how long this gap has been waiting to be filled.

3. Research Area and Technologies Applied

3.1 Key Research Area

This component sits squarely at the intersection of online machine learning and MLOps. The key research area is streaming anomaly detection with automated model lifecycle management.

Online machine learning is the discipline concerned with models that learn from data one sample at a time, updating continuously as new data arrives rather than training on a static batch and freezing. This is the natural paradigm for systems responding to live telemetry streams. The river library implements this paradigm through the learn_one and score_one interface, allowing

the Half Space Trees model to update in microseconds per event without ever storing the full dataset in memory.

The MLOps dimension addresses how models move through their lifecycle in production. A model is not a static artifact. It is a living system that requires monitoring, versioning, validation, and occasionally replacement. The novel research contribution of this component is the automated bridge between these two disciplines. The drift detector acts as the automated policy engine that decides when a model lifecycle transition should occur, replacing the human who would otherwise make that decision on a monthly schedule.

3.2 Technologies and Selection Rationale

The river library was chosen as the streaming ML framework because it implements Half Space Trees natively (the anomaly detection algorithm used throughout this project), implements ADWIN natively (the drift detection algorithm central to this component), and processes data in a single pass online fashion that is architecturally compatible with the FastAPI ingestion endpoint feeding it live telemetry. Alternatives like scikit multiflow are less actively maintained and PyOD is batch oriented. River is the clear choice.

ADWIN is the drift detection algorithm at the heart of this component. ADWIN maintains a variable length sliding window over the incoming stream of anomaly scores. The window shrinks or grows adaptively based on a statistical test derived from the Hoeffding bound. Specifically, ADWIN compares the mean anomaly score in the recent portion of the window against the mean in the older portion. When the difference between these means exceeds a threshold determined by the false positive rate parameter delta, ADWIN declares that drift has occurred. With delta set to 0.002 the false positive rate is bounded at 0.2 percent. This is a mathematically rigorous guarantee, not a heuristic threshold.

There are three categories of event that can trigger a model update or rollback in this system. Drift based triggering is the primary mechanism. When ADWIN fires, the system immediately initiates the full rollback flow. This handles the most common case of gradual distribution shift. Time based triggering handles the case where drift is slow enough that ADWIN does not fire but the model has nevertheless degraded over a long operational period. At a configurable interval the system schedules a re evaluation of model validity and a potential re warm up on freshly accumulated data. Event based triggering handles the case where a known change in the environment should prompt model re evaluation. When the Kubernetes event stream signals a new deployment in the target namespace, the system flags the model for immediate validity checking. New deployments are the single most common cause of baseline shift in microservices environments and waiting for ADWIN to detect the consequences is slower than acting on the deployment signal directly.

MLflow was chosen over Weights and Biases (primarily cloud hosted and proprietary) and DVC (focused on data versioning rather than model versioning) because it is fully open source, runs

entirely on premises within the Kubernetes cluster, and provides the Production and Archived stage labels that the rollback logic depends on.

The SMD (Server Machine Dataset) is the primary training and evaluation dataset. SMD contains 28 server machine entities, each with 38 telemetry features covering CPU, memory, network I/O, disk I/O, and derived application metrics. The dataset provides per timestamp ground truth anomaly labels across approximately 28,000 rows per entity. This combination of realistic multivariate server telemetry and labelled anomalies makes SMD ideal for both training the initial model and evaluating the dynamic updating pipeline against a static baseline (Su et al., 2019). The NAB (Numenta Anomaly Benchmark) dataset provides a secondary standardised scoring metric specifically designed for streaming anomaly detectors.

4. High Level Architecture and Self Evaluation Plan

4.1 Architecture Overview

The component operates in two phases. The warm up phase establishes the initial model. The full SMD training split for a given service entity is fed through the Half Space Trees model using learn_one only. No scoring occurs during warm up because the model has not yet seen enough data to produce meaningful anomaly scores. When warm up completes, the model state is serialised to a pickle file and logged as an MLflow artifact. This artifact is tagged as the Production model version and stored as the first stable snapshot that the rollback mechanism can restore to.

The inference and updating phase runs continuously. For each incoming metric payload from the Prometheus telemetry stream, the system calls score_one on the current feature vector to obtain an anomaly score between 0.0 and 1.0, then calls learn_one to update the model weights. The anomaly score is then fed into the ADWIN detector to monitor the score distribution in real time. If ADWIN does not fire, the event is published to Redis Streams and Redis Pub/Sub for downstream consumption by the Decision Engine and the React dashboard respectively. If ADWIN fires, the rollback flow is initiated before publication.

4.2 MLflow Rollback Flow

The rollback flow has five steps. In step one, the current drifted model state is serialised and logged to MLflow as a pre rollback artifact. This artifact is retained permanently for forensic analysis. In step two, the drifted run is tagged as Archived in the MLflow model registry with the step number, drift score, and rolling mean anomaly score recorded as metadata. In step three, the system retrieves the last Production tagged model artifact from MLflow and deserialises it into memory. In step four, the restored model replaces the current in memory model for all subsequent scoring calls with zero downtime because the swap is atomic in Python. In step five, the rollback event is published to the Redis Pub/Sub channel ws:anomaly_broadcast, which

pushes a real time drift alert notification to the React dashboard and makes the recovery immediately visible to operations teams.

4.3 Self Evaluation Plan

The self evaluation plan compares a dynamic model against a static baseline on the same SMD test stream. The static baseline is the model frozen after warm up, receiving no further updates and no drift detection. The dynamic model runs with ADWIN monitoring and all three trigger types active.

Both models are evaluated against the same ground truth anomaly labels at regular checkpoints through the test stream. This allows the evaluation to demonstrate how the accuracy gap between the two models evolves over time. The primary metrics are precision, recall, and F1 score at each checkpoint. A result where the dynamic model maintains consistently higher F1 than the static baseline across the full test period validates the core hypothesis that continuous lifecycle management improves long term model performance.

A secondary evaluation uses the NAB scoring framework, which rewards early detection of anomaly windows and penalises false positives, to produce a standardised benchmark score comparable to published results from other streaming detectors. The target NAB score is above 0.55, which represents competitive performance relative to published state of the art results in the NAB benchmark leaderboard (Numenta, 2016).

5. User and Functional Requirements

FR1	The system must detect distribution shifts in anomaly score streams automatically without human monitoring or manual intervention. This is the core requirement that this component exists to fulfil.
FR2	The system must support three independent and simultaneously active model update trigger types (drift based via ADWIN, time based via scheduled re evaluation, and event based via Kubernetes deployment events).
FR3	The system must roll back to a verified stable model version within one scoring cycle of a drift detection event, ensuring model quality is restored before the next incoming telemetry event is processed.
FR4	The system must log every model state transition to MLflow with full context including step number, drift score, rolling mean anomaly score, timestamp, and reason for the transition.

FR5	The system must operate in a multi replica FastAPI deployment without shared in memory state. All model state must be either local to the replica or retrievable from shared persistent storage via the MLflow artifact store.
FR6	The system must add no more than 5 milliseconds of additional latency to the per request scoring pipeline on the target hardware (i7-1165G7 processor, 16 GB RAM).
FR7	The system must maintain a minimum F1 score of 0.65 on the SMD test dataset throughout a simulated five week operational period that includes injected concept drift events.

6. Work Breakdown Structure

The following table presents the planned work breakdown across fourteen weeks. Each phase builds on the previous and the integration phases at the end are reserved for system validation and final testing.

Weeks	Tasks	Deliverable
1 to 2	Literature review. Concept drift algorithm study. Development environment setup including Python 3.11 virtual environment, river, and MLflow.	Annotated bibliography. Configured dev environment.
3 to 4	SMD dataset acquisition and preprocessing pipeline. Normalisation module fitted on training split only. Parquet caching. Row by row streaming generator.	data_pipeline.py module. Preprocessed SMD Parquet files.
5 to 6	HST warm up phase implementation. Core score and learn inference loop. Initial ADWIN integration with configurable delta parameter.	hst_model.py module. Functional warm up and inference pipeline.
7 to 8	MLflow lifecycle module. Experiment initialisation, run management, metric logging, stable snapshot checkpointing, and five step rollback flow.	mlflow_lifecycle.py module. Verified rollback on simulated drift.
9 to 10	Three trigger types implementation. Time based and event based triggers added to existing drift based trigger. Trigger policy configuration layer.	Complete trigger system. Config via environment variables.
11 to 12	Evaluation framework. Static baseline vs dynamic model comparison pipeline. NAB scoring integration. Precision, recall, and F1 over time visualisation.	Evaluation scripts. Benchmark results vs static baseline.
13 to 14	FastAPI integration. Redis Pub/Sub drift alert publishing. End to end integration testing. Documentation and code review.	Integrated component. Full documentation. Integration test suite.

7. Commercialization and Business Potential

Model decay is not a research curiosity. It is a documented, expensive, and widespread problem in production AI systems. A 2022 Gartner survey estimated that up to 85 percent of AI and ML projects fail to move beyond proof of concept into production, and model degradation is consistently cited among the top three reasons for project abandonment after initial deployment. Every organisation running a production ML system faces this problem, and most of them are solving it with manual retraining schedules that require dedicated engineering time every month.

The immediate commercial opportunity is a drop in module for any Python based streaming ML deployment. The component is designed around the river and MLflow APIs, both of which are widely adopted in the industry. A company running any river based anomaly detector or classifier could integrate this lifecycle management layer with minimal changes to their existing infrastructure. The value proposition is clear. Automated model lifecycle management replaces a recurring manual engineering task with a software system that runs continuously and costs nothing beyond the initial integration effort.

The target market spans several high value verticals. Financial services companies use anomaly detection for fraud and transaction monitoring, where model decay directly translates to missed fraud and increased losses. E-commerce platforms use it for demand forecasting and inventory management, where a degraded model produces poor recommendations that reduce revenue. Industrial IoT operators use anomaly detection for equipment health monitoring, where missed detections cause unplanned downtime and safety incidents. All of these verticals share the same underlying model decay problem.

A SaaS commercialisation pathway exists as a managed MLOps service specifically for streaming ML systems. Existing MLOps platforms focus exclusively on batch model retraining pipelines. No major commercial platform offers first class support for online learning lifecycle management. This is a genuine and unoccupied position in the commercial market that directly mirrors the research gap identified in Section 1.

The open source release of this component as a standalone library provides a foundation for community adoption and commercial support contracts. The architecture is explicitly modular. The drift detection layer, the trigger policy layer, and the MLflow integration layer are independently replaceable, allowing organisations to substitute their own preferred tools while retaining the overall lifecycle management logic. This modularity extends the addressable market beyond river and MLflow users to any Python ML deployment that faces the model decay problem.

References

[1] A. Bifet and R. Gavalda, Learning from Time Changing Data with Adaptive Windowing, in Proc. SIAM International Conference on Data Mining, 2007, pp. 443 to 448.

[2] J. Montiel et al., River: Machine Learning for Streaming Data in Python, Journal of Machine Learning Research, vol. 22, no. 110, pp. 1 to 8, 2021.

[3] M. Zaharia et al., Accelerating the Machine Learning Lifecycle with MLflow, IEEE Data Engineering Bulletin, vol. 41, no. 4, pp. 39 to 45, 2018.

[4] Y. Su et al., Robust Anomaly Detection for Multivariate Time Series through Stochastic Recurrent Neural Network, in Proc. ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2019, pp. 2828 to 2837.

[5] J. Gama et al., A Survey on Concept Drift Adaptation, ACM Computing Surveys, vol. 46, no. 4, pp. 1 to 37, 2014.

[6] S. Lu et al., Learning under Concept Drift: A Review, IEEE Transactions on Knowledge and Data Engineering, vol. 31, no. 12, pp. 2346 to 2363, 2019.

[7] J. Soldani, D. A. Tamburri, and W. J. Van Den Heuvel, The pains and gains of microservices: A systematic grey literature review, Journal of Systems and Software, vol. 146, pp. 215 to 232, 2018.

[8] D. Cerny et al., Microservices: A review of concepts, challenges, and solutions, IEEE Access, vol. 8, pp. 11 to 35, 2020.

[9] Numenta, The Numenta Anomaly Benchmark (NAB), GitHub repository, 2016. Available at https://github.com/numenta/NAB

[10] OmniAnomaly, Server Machine Dataset, GitHub repository, 2019. Available at https://github.com/NetManAIOps/OmniAnomaly